Assignment 3: System Calls

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 2 §2.3–2.4, pp. 55–66 [2]; Bovet & Cesati, *Understanding the Linux Kernel*, Ch. 10, pp. 398–419 [1].

Concepts That Click

3.1 Conceptual

Q1. A classmate posts: "`printf` is a system call, `fork` is really file management because the child opens files, and the six categories are cosmetic labels." Respond precisely [2].

(a) Trace `printf("hello\n")` from the C library to the kernel. Name the user-space buffer, the system call the library eventually issues, and explain why the same `printf` may cause zero, one, or many system calls.

(b) State whether `printf` is a system call, and contrast the programmer's API view with the kernel's system-call view in terms of layer, buffering, portability, and return value.

(c) Explain what would go wrong if the kernel were linked into every user program. Address protection, isolation, portability, and evolution, and name the hardware mechanism that makes the boundary enforceable.

(d) Classify `fork`, `open`, `ioctl`, `getpid`, `pipe`, and `chmod` into the six categories. Then explain why `fork` is process control rather than file management even though the child may open files.

Q2. Trace `write(fd, buf, n)` end-to-end from the library stub to the return to user mode [1].

(a) List the steps in order. Name the argument registers, the register holding the system-call number, the trap instruction on x86-64 and on legacy x86, what the hardware saves, the mode switch, the system-call table, the handler, and the return register.

(b) Explain why the kernel dispatches through a table indexed by number rather than searching for a handler by name, and what it means that the number is part of the *application binary interface* (ABI). A maintainer proposes renumbering `write` from 1 to 7 to make room for a new call: state exactly what breaks, and why new calls are appended instead.

(c) At which step are `fd` and `buf` validated? What does the kernel do if `buf` points into kernel memory, and why may it not simply dereference the pointer? Name the error code and the Linux copy routines it uses instead.

3.2 Numerical

Q3. Following the lecture's `cp` trace, a shell runs `cp big.bin copy.bin` [2]. The source is exactly 8 MiB and `cp` uses an 8 KiB user buffer. The shell `fork`s, the child `execs` `cp`, `cp` opens both files, loops `read`/`write`, closes both descriptors, and calls `exit(0)`; the shell then `wait`s. Assume no other system calls, no signals, and no page faults.

(a) Count the system calls. Separate the shell's `fork`/`exec`/`wait`, `cp`'s two `open`s, two `close`s, and one `exit`, the `read` calls (include the final `read` that returns 0), and the `write` calls. Give the total and the number of user↔kernel mode switches, one per system call.

(b) Give the number of bytes moved from file to user buffer and from user buffer to file over the whole run, and the total crossing the user/kernel boundary. Contrast this with the 8 MiB of unique file data.

(c) A system call costs 0.9 μs. Compute the total system-call overhead in milliseconds, and express it as a percentage of the 8 MiB transfer time at 200 MiB/s.

(d) Repeat (a) and (c) with a 64 KiB buffer. State the reduction factor in mode switches.

Q4. Batching changes the number of boundary crossings, not the number of bytes.

(a) A 1 MiB file is read once with `read(fd, buf, 1048576)`, versus with exactly 1,048,576 successful one-byte reads (ignore the terminating end-of-file read for this comparison). Each call costs 0.75 μs of kernel overhead independent of the byte count. Compute the total system-call overhead in each case, giving the one-byte total in milliseconds and seconds, and the ratio between them. At 400 MiB/s, compute the data-transfer time; in the one-byte case, compute total elapsed time and the percentage spent crossing the boundary. State the lesson in one sentence.

(b) The x86-64 Linux convention reserves `rax` for the system-call number and provides six argument registers: `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. A proposed call takes nine 8-byte arguments.

(i) How many arguments fit in the argument registers?

(ii) If the remaining arguments go on the user stack, how many bytes must the kernel read from the stack?

(iii) If instead all nine arguments are placed in one parameter block and a single register points to it, how many bytes is the block, and how many argument registers are freed?

(iv) When a call needs more arguments than the registers can carry, which method does a real ABI choose, and why?

3.3 Design

Q5. A shell executes `cat notes.txt | wc -l`. Design the complete system-call trace, beginning in the shell before the first `fork` [2].

Concepts That Click

(a) Give the ordered trace. For each call, name the process (shell, `cat`, or `wc`), the category, the arguments, and what the return value is used for. Include `pipe`, both `fork`s, the descriptor wiring (`dup2` and the closes), both `exec`s, the data path, and the final `wait`s.

(b) Explain why the write end must be closed in `wc` and the read end in `cat`, and state exactly what happens to `wc` if `cat`'s write end is never closed.

(c) Count the system calls for a 96 KiB file read in 32 KiB chunks, assuming `cat` and `wc` both use 32 KiB buffers and both read until end-of-file. Count closes that keep a pipe end open; exclude optional closes of standard descriptors that `exit` releases anyway. State which counts depend on the file size.

Q6. A 64-bit architecture reserves `rax` for the system-call number and provides six argument registers (`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`) [1]. You are adding a kernel-assisted copy call with seven logical arguments:

```
int copy_range(int fd_in, off_t *off_in, int fd_out,
        off_t *off_out, size_t len, unsigned int flags,
        size_t *out_copied);
```

(a) Explain why registers alone are insufficient. Give the exact count of arguments and registers.

(b) Design a parameter block in the caller's memory. Give a table with offset, size, field, and direction (in, in-out, or out), then state the block's total size and alignment and what goes in `rdi` and `rax`.

(c) Give the kernel's validation sequence, in order, for the block pointer and for each pointer field. Name `EFAULT` and the `copy_from_user`/`copy_to_user` discipline. Say what a `NULL` `off_in` means versus an invalid non-`NULL` one.

(d) The call copies 4 MiB and succeeds. How many bytes of argument data are copied into the kernel and how many back out? How many user/kernel mode switches occur, and does the 4 MiB payload add any?

(e) If all seven arguments were pushed on the user stack instead, how many bytes would the kernel read from the stack? Why is the block still the better choice for this call?

Concepts That Click

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.